

Lucrarea nr. 2: **Implementarea funcțiilor binare cu rețele de porți logice****1. Scopul lucrării**

În această lucrare se face o prezentare sintetică a principalelor etape ce trebuie parcurse în procesul de implementare a funcțiilor binare cu ajutorul porților logice.

Prima parte a lucrării prezintă abordarea clasică (bazată pe utilizarea circuite integrate digitale de complexitate redusă) iar partea a doua prezintă abordarea modernă (bazată pe utilizarea de structuri logice reconfigurabile de tip CPLD). În strânsă legătură cu partea a doua a lucrării, se prezintă modul în care un circuit sau sistem digital poate fi implementat într-o structură reconfigurabilă de tip CPLD cu ajutorul mediului de dezvoltare ISE-WebPack, produs de firma Xilinx. Totodată sunt prezentate primele noțiuni despre limbajul VHDL, urmând ca acestea să fie aprofundate în lucrările viitoare.

2. Considerente teoretice**2. 1. Funcția de transfer a unui circuit logic combinațional (CLC)**

Pentru cazul general al unui CLC cu n intrări și m ieșiri, starea semnalelor de intrare este specificată printr-un vector cu n componente $X(x_1, x_2, \dots, x_n)$, iar starea ieșirilor de un vector cu m componente $Y(y_1, y_2, \dots, y_m)$. În mod evident, fiecare componentă a vectorului de intrare și a celui de ieșire este binară (poate avea valoarea 0 sau 1).

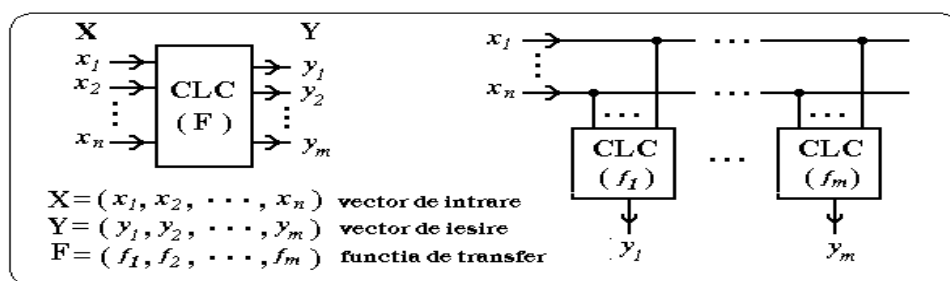


Fig. 1. Schema bloc a unui circuit logic combinațional

Relația de legătură dintre intrare și ieșire este descrisă de către funcția de transfer $F(f_1, f_2, \dots, f_m)$. În cazul particular în care $m=1$, funcția de transfer are o singură componentă și este denumită **funcție binară**. Studiul funcțiilor binare prezintă o importanță deosebită deoarece, conform reprezentării din fig. 1, un CLC cu m ieșiri poate fi descompus într-o sumă de m funcții binare.

Funcția de transfer a unui CLC poate fi descrisă în mai multe moduri dintre care cele mai utilizate sunt: **tabelul de adevăr**, **formele canonice** (conjunctivă sau disjunctivă), **diagrama Veitch-Karnaugh**, **formele analitice reduse** (fie pentru valoarea de 1, fie pentru valoarea de 0), **descrierea în limbaj natural**, etc. Pentru cei interesați, detalii suplimentare despre aceste forme de reprezentare se regăsesc în anexa 2.

2. 2. Sinteza funcțiilor binare folosind rețele de porți logice – Metoda clasică

Realizarea practică a funcțiilor binare se poate face în mai multe moduri: folosind porți logice, folosind circuite de complexitate medie (multiplexoare sau demultiplexoare), folosind memorii ROM sau EEPROM, folosind structuri logice programabile (PAL, GAL CPLD sau chiar FPGA), etc.

În această secțiune a lucrării se prezintă implementarea bazată pe porți logice. Succesiunea și denumirea etapelor necesare în procesul de sinteză sunt prezentate în tabelul 1.

La rândul său, implementarea cu porți logice, poate avea mai multe variante:

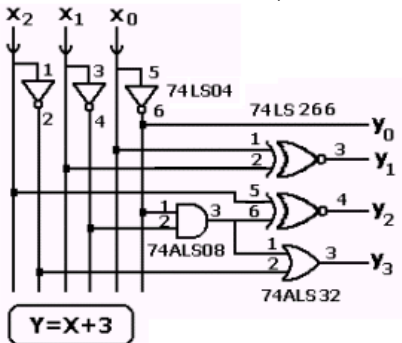
- folosind **logică combinată** – se permite utilizarea oricărui tip de poartă logică;
- folosind **logică de același tip** – se permite utilizarea unui singur tip de poartă logică, fie NAND, fie NOR;

2.2.1. Logica combinată

Tabelul 1

Nr. etapă	Denumire etapă	Exemplu
1	Definirea foarte exactă a funcției logice ce trebuie realizate. Se pleacă de la descrierea în termeni naturali a funcționării circuitului.	Ce trebuie să facă circuitul ? Adună la numărul de intrare X , constanta zecimală 3. Așadar, funcția de transfer a CLC-ului este: $Y = X + 3$. Starea logică a intrărilor este interpretată ca fiind scrierea binară a numărului de intrare X , iar starea logică a ieșirilor se consideră a fi scrierea binară a numărului Y . Câte intrări și câte ieșiri are circuitul ? Din datele inițiale știm că are 3 intrări. Numărul de ieșiri trebuie deduse ținând seama de funcția realizată de circuit. Cel mai mare număr exprimat pe 3 biți este $X=7$, caz în care $Y=7+3=10$. Rezultă că circuitul

		trebuie să prezinte 4 ieșiri.																																																																																	
2	Alegerea unei modalități de descriere a funcției logice. În principiu, se poate alege orice metodă de reprezentare cunoscută.	Pentru acest circuit, cea mai adecvată metodă de reprezentare este dată de tabelul de adevăr complet. <table><tr><td>X</td><td>x₂</td><td>x₁</td><td>x₀</td><td>y₃</td><td>y₂</td><td>y₁</td><td>y₀</td><td>Y</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>3</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>4</td></tr><tr><td>2</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>5</td></tr><tr><td>3</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>6</td></tr><tr><td>4</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>7</td></tr><tr><td>5</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>8</td></tr><tr><td>6</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>9</td></tr><tr><td>7</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>10</td></tr></table>	X	x ₂	x ₁	x ₀	y ₃	y ₂	y ₁	y ₀	Y	0	0	0	0	0	0	1	1	3	1	0	0	1	0	1	0	0	4	2	0	1	0	0	1	0	1	5	3	0	1	1	0	1	1	0	6	4	1	0	0	0	1	1	1	7	5	1	0	1	1	0	0	0	8	6	1	1	0	1	0	0	1	9	7	1	1	1	1	0	1	0	10
X	x ₂	x ₁	x ₀	y ₃	y ₂	y ₁	y ₀	Y																																																																											
0	0	0	0	0	0	1	1	3																																																																											
1	0	0	1	0	1	0	0	4																																																																											
2	0	1	0	0	1	0	1	5																																																																											
3	0	1	1	0	1	1	0	6																																																																											
4	1	0	0	0	1	1	1	7																																																																											
5	1	0	1	1	0	0	0	8																																																																											
6	1	1	0	1	0	0	1	9																																																																											
7	1	1	1	1	0	1	0	10																																																																											
3	Alegerea modalității (sau a tehnologiei) de implementare.	Optăm pentru utilizarea porților logice deoarece scopul acestei lucrări constă în prezentarea acestei metode. În această etapă trebuie avute în vedere aspecte precum: - performanțele propuse (viteză de lucru, consum de energie, etc.); - costul circuitelor utilizate; - seria în care se va produce circuitul, etc.																																																																																	
4	Simplificarea funcției logice. Această etapă nu este strict necesară pentru toate modalitățile de implementare a funcțiilor logice. Pentru implementarea cu porți, această etapă este necesară deoarece reduce numărul de circuite necesare. Proprietățile algebrei binare trebuie aplicate astfel încât să obține expresii finale cât mai simple posibil.	Avem de implementat 4 funcții binare ce depind de aceleași variabile de intrare. ♦ Ieșirea y₀. Din analiza tabelului de adevăr se observă că avem $y_0 = \bar{x}_0$ ♦ Ieșirea y₁. Folosind prima formă canonică obținem: $y_1 = \begin{vmatrix} \bar{x}_2\bar{x}_1\bar{x}_0 \\ \bar{x}_2x_1x_0 \\ x_2\bar{x}_1\bar{x}_0 \\ x_2x_1x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 & \bar{x}_1\bar{x}_0 \\ x_2 & x_1x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 & \bar{x}_1\bar{x}_0 \\ x_2 & x_1x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 & \bar{x}_1\bar{x}_0 \\ x_2 & x_1x_0 \end{vmatrix}$ ♦ Ieșirea y₂. Folosind prima formă canonică obținem: $y_2 = \begin{vmatrix} \bar{x}_2\bar{x}_1x_0 \\ \bar{x}_2x_1\bar{x}_0 \\ \bar{x}_2x_1x_0 \\ x_2\bar{x}_1\bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 & \bar{x}_1x_0 \\ x_2 & x_1\bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 & x_0 \\ x_2 & x_1 \end{vmatrix} = \begin{vmatrix} \bar{x}_2P \\ x_2P \end{vmatrix}; \text{ unde } P = \bar{x}_1\bar{x}_0$ ♦ Ieșirea y₃. Folosind a doua formă canonică obținem: $y_3 = \begin{vmatrix} \bar{x}_2\bar{x}_2\bar{x}_2 \\ x_1\bar{x}_1\bar{x}_1 \\ \bar{x}_0x_0\bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \\ x_1\bar{x}_1 \\ \bar{x}_0x_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \\ \bar{x}_1\bar{x}_0 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 \\ P \end{vmatrix}$ Observații: - Funcția y₁, se implementează cu o poartă sau exclusiv negat; - Produsul P, apare în expresia a două funcții binare, el va fi calculat o singură dată și apoi folosit în sinteza ambelor funcții;																																																																																	
5	Deducerea schemei logice. Atenție: Există o mulțime de scheme logice pentru același tabel de adevăr! Schema logică depinde foarte mult de modul în care s-au aplicat proprietățile algebrei binare în procesul de simplificarea expresiei algebrice.	Schema logică rezultă din relațiile obținute la punctul anterior. Se observă că această implementare necesită 7 porți logice dar acestea se găsesc în 4 circuite integrate diferite.																																																																																	
6	Implementarea hardware. În această etapă, după	Alegem familia 74LS***. Consultând un catalog de firmă găsim următoarele coduri: - pentru inversoare: 74LS04; - pentru poarta AND cu două intrări: 74LS08;																																																																																	

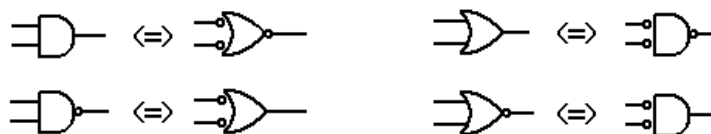
	ce s-a stabilit familia de circuite logice, se identifică codurile circuitelor ce conțin porțile ce ne interesează, apoi se face alocarea pinilor. După terminarea acestei etape avem toate datele necesare pentru a trece la realizarea sa practică.	- pentru poarta sau exclusiv negat: 74LS266; - pentru poarta sau cu două intrări: 74LS32;  Se observă că am avut nevoie de 4 circuite integrate însă gradul lor de utilizare este următorul: 1/2 din 74LS04, 1/2 din 74LS266, 1/4 din 74ALS08 și 1/4 din 74ALS32.
--	---	---

2.2.2. Logica de același tip

Așa după cum se știe, sistemele de operatori: (AND, NOT), (OR, NOT), (NOR), (NAND) sunt sisteme complete de operatori - ceea ce înseamnă că pot fi utilizate în descrierea oricărei funcții binare. Interes practic mai mare prezintă ultimele două deoarece ne indică posibilitatea de a implementa orice funcție logică cu ajutorul unui singur tip de poartă logică, fie NAND fie NOR.

Metoda grafică. Transpunerea unei scheme logice oarecare într-o logică de același tip se face, utilizând în mod convenabil, următoarele reguli:

- **R1:** La ieșirea unui etaj logic se poate introduce un cerculeț de negare dacă, corespunzător, la intrarea nivelului logic următor se face același lucru ("dubla negare este adevăr").
- **R2:** Introducerea la o ieșire a unui cerculeț de negare trebuie urmată de adăugarea unui inversor. Introducerea la o intrare a unui cerculeț de negație trebuie precedată de negarea respectivei variabile de intrare.
- **R3:** Trecerea de la o poartă la alta se face conform echivalențelor de mai jos (deduse din teoremele lui DeMorgan):



În funcție de schema logică inițială rezultatul transpunerii în logică de același tip ne poate conduce spre scheme mai simple sau, dimpotrivă, mai complicate decât schema inițială. Câteva transpuneri în logică de același tip sunt exemplificate în figura 6.

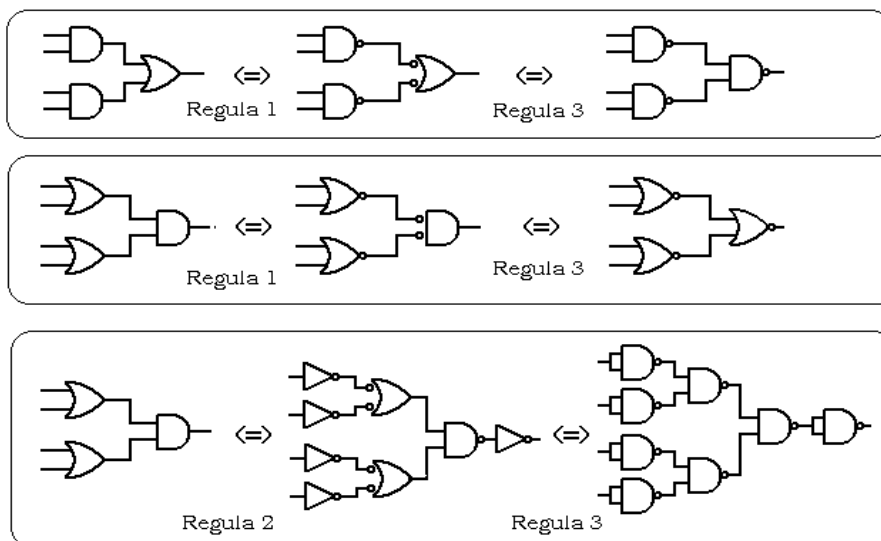


Fig. 6. Transpunerea unor scheme logice în logică de același tip

Pentru situațiile în care logica de același tip nu se impune în mod strict, aplicarea regulilor R1, R2, R3, pe porțiuni convenabile ale schemei, poate să determine reducerea necesarului de porți al schemei. De exemplu, aplicarea regulii R3

pentru funcția y_2 , are ca efect reducerea necesarului de porți de la 4 la 2 și, în plus, implementarea se face folosind două tipuri de porți față de trei câte era inițial. Această situație este prezentată în figura 7.

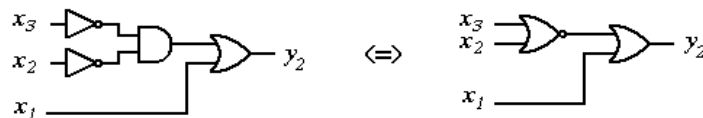


Fig. 7. Reducerea numărului de porți folosind echivalența dintre acestea

Metoda analitică. Această metodă presupune aplicarea repetată și în mod convenabil a teoremelor lui DeMorgan pentru o expresie inițială a funcției logice. Dacă se dorește implementarea în logică de tip NAND-NAND, se urmărește transformarea operațiilor de adunare în operații de înmulțire, iar dacă se dorește implementarea în logică de tip NOR-NOR, se urmărește transformarea operațiilor de înmulțire în operații de adunare.

În tabelul 2, se prezintă modul de trecere la logica de tip NAND-NAND a circuitului logic prezentat în tabelul 1.

Tabelul 2

Trecerea la logica de tip NAND-NAND	Schema logică/Observații
Se pornește de la expresiile reduse obținute în tabelul 1. $y_0 = \bar{x}_0$ $y_1 = \left[\begin{array}{c c} \bar{x}_1 & \bar{x}_0 \\ \hline x_1 & x_0 \end{array} \right] = \overline{\bar{x}_1 \bar{x}_0} \quad \overline{x_1 x_0}$ $y_2 = \left[\begin{array}{c c} \bar{x}_2 & x_0 \\ \hline x_2 & \bar{x}_1 \end{array} \right] = \left[\begin{array}{c c} \bar{x}_2 & \bar{x}_1 \bar{x}_0 \\ \hline x_2 & \bar{x}_1 \bar{x}_0 \end{array} \right] = \overline{\bar{x}_2 \bar{x}_1 \bar{x}_0} \quad \overline{x_2 \bar{x}_1 \bar{x}_0}$ $y_3 = \left[\begin{array}{c c c} \bar{x}_2 & \bar{x}_2 & \bar{x}_2 \\ \hline x_1 & \bar{x}_1 & \bar{x}_1 \\ \hline \bar{x}_0 & x_0 & \bar{x}_0 \end{array} \right] = \left[\begin{array}{c c} \bar{x}_2 & \bar{x}_1 \bar{x}_1 \\ \hline \bar{x}_0 & x_0 \bar{x}_0 \end{array} \right] = \left[\begin{array}{c c} \bar{x}_2 & \bar{x}_1 \bar{x}_0 \end{array} \right] = \overline{\bar{x}_2 \bar{x}_1 \bar{x}_0}$	• Această realizare necesită 9 porți NAND cu 2 intrări și o poartă NAND cu trei intrări. Așadar, avem nevoie de 4 circuite: 3 circuite 74LS00 și un circuit de tip 74LS10. • Există posibilitatea de a realiza schema logică numai cu trei circuite integrate dacă se intervine asupra modului de implementare al funcției y_2, astfel încât să folosim doar porți NAND cu două intrări.

2. 3. Sinteza funcțiilor binare folosind structuri logice programabile de tip CPLD

CPLD-urile sunt structuri logice programabile, mai corect spus reconfigurabile, care într-o singură capsulă înglobează o mulțime de resurse hardware digitale. Spre exemplu, circuitul din machetele de laborator, XC95108, conține echivalentul a cca. 2000 de porți convenționale. Există însă structuri programabile mult mai complexe.

Realizarea de aplicații folosind astfel de circuite nu se mai face manual, fiecare firmă producătoare de circuite reconfigurabile asigură și un mediu software integrat care ușurează foarte mult dezvoltarea aplicațiilor. De regulă, aceste medii software acceptă mai multe modalități de descriere a funcționării circuitului (sistemului) logic. Cele mai întâlnite modalități de descriere sunt: folosirea de scheme logice; folosirea de fișiere VHDL; folosirea de diagrame de tranziție a stărilor; etc.

În desfășurarea acestei lucrări se folosește mediul de dezvoltare ISE - WebPack, produs de firma Xilinx, iar ca metode de descriere a funcționării circuitului folosim fie schemele logice, fie limbajul de descriere hardware VHDL.

2.3.1. Descrierea circuitelor combinaționale cu ajutorul editorului de scheme

Pentru a realiza o aplicație în mediul **ISE-WebPack** pe baza editorului de scheme, este necesar să parcurgem următoarele etape:

- **Deschiderea unui nou proiect.** Pentru aceasta este necesară parcurgerea următoarei secvențe de acțiuni: se alege **File** → **New Project** → în fereastra apărută se specifică **numele** proiectului (spre exemplu **circuit_sumare**), **locul** unde acesta va fi salvat și proprietățile proiectului → **OK**;
- **Adăugarea unui fișier sursă conținând schema logică (fișier de tip .sch).** Din fereastra **Project Navigator** se alege **Project** → **New Source** → în fereastra apărută se alege **Schematic**, se specifică denumirea noului fișier (spre exemplu **abc**) → **Next** → **Finish**. După aceste comenzi, în fereastra **Source in Project** se va adăuga fișierul **abc.sch**, apoi, în mod automat se lansează în execuție editorul de scheme **Xilinx ECS**.

- **Desenarea schemei logice.** Deoarece modul de editare a schemei logice este similar oricărui editor de scheme electrice, nu vom insista asupra acestui aspect. În final trebuie avut grijă ca schema logică să fie salvată.
- **Adăugarea fișierului de constrângeri** (fișier de tip **.ucf**). Acest fișier este foarte important deoarece prin intermediul sau sunt precizați pinii circuitului CPLD unde vor fi conectate intrările și ieșirile circuitului proiectat de noi. Pentru adăugarea acestui fișier se procedează astfel: din fereastra **Project Navigator** se alege **Project** → **New Source** → în fereastra apărută se alege **Implementation Constrain File**, se specifică denumirea noului fișier (spre exemplu **cons_sumator**) → **Next** → în fereastra apărută se alege fișierul sursă căruia i se asociază acest fișier de constrângeri (în cazul de față avem o singură sursă **abc.sch**) → **Next** → **Finish**. În urma acestor operații în fereastra surselor implicate în proiect (**Source in Project**) apare o nouă componentă **cons_sumator.ucf**.

Dacă se face dublu clic pe **abc.ucf** se lansează un utilitar care ne ajută să edităm fișierul de constrângeri. Din fereastra apărută se selectează tabul **Ports**. După această alegere, pe ecran apare lista porturilor de intrare și de ieșire ale sistemului digital proiectat de noi. Pentru fiecare port trebuie să specificăm pinul CPLD-ului unde dorim conectarea respectivului port, aceasta presupune completarea coloanei denumită **Location**. După completare se face o salvare a fișierului de constrângeri.

Deoarece fișierul de constrângeri este de tip text, există și posibilitatea editării manuale a acestui fișier prin următoarea secvență de comenzi: din fereastra **Source in Project** se alege fișierul ce descrie funcționarea circuitului proiectat de noi (în cazul de față **abc.sch**) → din fereastra **Processes for Current Source** se face dublu clic pe opțiunea **Edit Constrains**. După aceste comenzi apare o fereastră în care se introduc constrângerile, cu respectarea sintaxei.

Trebuie avut grijă ca numărul pinului completat în coloana Location să corespundă cu schema hardware a machetei de laborator. Pentru acesta este absolut obligatoriu să consultați tabelul de conexiuni prezentat în anexe.

- **Implementarea circuitului în CPLD.** După introducerea surselor în proiect (etapă denumită **Design Entry**), urmează câteva etape ce se desfășoară în mod automat. Ultima etapă constă în generarea fișierelor de configurare a circuitului CPLD, etapă denumită inpropriu de generare a fișierelor de programare (**Generate Programming Files**). Pentru generarea fișierului de configurare, în fereastra resurselor se alege fișierul principal (**abc.sch**) iar în fereastra proceselor disponibile pentru acest fișier se face dublu clic pe opțiunea **Configure Device (iMPACT)**. În ferestrele următoare se aleg următoarele comenzi: **Configure Devices** → **Next** → **Boundary Scan Mode** → **Next** → **Automatically connect to cable** → **Finish** → **OK** → se alege fișierul **abc.jed** → **Open** → clic dreapta pe icoana circuitului XC95108 → **Program...** → se alege opțiunea **Erase Before Programming** → **OK**. Dacă totul este în regulă, după câteva secunde va apare mesajul **Programming Succeeded** și se poate trece la verificarea funcționării circuitului.

2.3.2. Descrierea circuitelor combinaționale cu ajutorul limbajului VHDL

Limbajul VHDL a fost conceput în vederea simulării și sintezei cât mai ușoare a unui circuit/sistem logic. La ora actuală, limbajul VHDL este considerat un limbaj suficient de puternic pentru descrierea unor sisteme digitale complexe dar și suficient de ușor de asimilat pentru cei care sunt familiarizați cu un program de nivel înalt precum Pascal sau C.

Proiectarea și sintetizarea unui sistem digital în limbajul VHDL presupune deschiderea unui proiect în care sunt implicate mai multe categorii de fișiere sursă. Mediul ISE-WebPack are posibilitatea de a sintetiza scheme logice pornind de la fișiere sursă de tip VHDL. Modul de lucru este similar cu cel prezentat în secțiunea anterioară, cu singura deosebire că atunci când se introduce o nouă sursă în proiect, în loc de opțiunea **Schematic** se alege **VHDL Module**.

Descrierea unui circuit logic cu ajutorul VHDL presupune utilizarea unei entități (**entity**) la care trebuie asociată în mod obligatoriu cel puțin o arhitectură (**architecture**).

Entitatea este folosită în principal pentru a specifica conexiunile exterioare, denumite porturi, prin specificarea numelui, a direcției datelor, a tipului de date vehiculate, etc.

Arhitectura este folosită pentru descrierea comportamentului intern al circuitului ce urmează a fi proiectat, cu alte cuvinte conține o descriere a funcționării circuitului sau schema logică a acestuia.

Înainte de a trece la prezentarea exemplelor facem precizarea că limbajul VHDL permite descrierea circuitelor logice în mai multe moduri:

- **descriere structurală** - proiectantul impune schema logică iar sarcina mediului software este de a "amplasa" corect această schemă în resursele interne ale circuitului în care se dezvoltă aplicația;
- **descriere comportamentală** - proiectantul face o descriere de nivel înalt a circuitului/sistemului urmând ca sinteza propriu-zisă a schemei logice să rămână în sarcina mediului software.
- **metode combinate** - proiectantul poate să opteze pentru descriere structurală pentru anumite blocuri funcționale și descriere comportamentală pentru altele.

O sursă VHDL este un fișier text în care, în secțiunea de descriere a funcționării circuitului (în arhitectură), găsim o listă formată din **declarații concurente** și **declarații secvențiale**. Ajunși aici trebuie să precizăm că ceea ce în programare se cheamă **instrucțiune**, în descrierea hardware a circuitelor poate denumirea de **declarație** (statement). Aceasta nu este singura diferență între programare și descrierea hardware, deosebirile de substanță, după cum vom vedea imediat, sunt date de modul în care sunt executate cele două tipuri de declarații.

Toate declarațiile concurente sunt executate simultan. Acest lucru se explică prin faptul că, în urma sintezei, fiecare declarație concurentă este implementată de un circuit hardware separat (implementare paralelă). Din acest motiv ordinea în care aceste declarații apar în listingul arhitecturi nu are nici o importanță.

Declarațiile secvențiale trebuie executate una după alta, într-o ordine bine stabilită, și numai după ce anumite condiții de declanșare au fost îndeplinite. În limbajul VHDL, pentru a separa cele două tipuri de declarații se folosește noțiunea de proces (**process**). În interiorul unui proces sunt incluse numai operațiile secvențiale ce trebuiesc executate atunci când anumite condiții de declanșare sunt îndeplinite. Condițiile de declanșare a procesului, altfel spus de declanșare a unei serii de acțiuni cu succesiune precisă, sunt specificate într-o așa zisă listă de sensivitate a procesului. Așadar, în interiorul procesului, ordinea declarațiilor este foarte importantă, ea indică succesiunea în care operațiile se execută.

Din punct de vedere al execuției, fiecare proces are același statut cu o declarație concurentă, cu alte cuvinte, toate procesele și toate declarațiile concurente se execută în același timp, deci sunt implementate în hardware prin scheme distincte.

În final precizăm că, în mod curent, procesele sunt folosite pentru implementarea circuitelor secvențiale și uneori pentru circuite combinaționale iar declarațiile concurente doar pentru circuite combinaționale.

Modul de utilizare al celor două tipuri de declarații precum și multitudinea de posibilități puse la dispoziție de limbajul VHDL se poate urmări în exemplele ce urmează, exemple ce au ca scop implementarea unei funcții binare de trei variabile având tabelul de adevăr prezentat în tabelul 3.

Tabelul 3

c	b	a	yout
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

◆ **Exemplul 1:** Descrierea funcției logice **yout** pornind de la tabelul de adevăr

Observații	Codul VHDL
Secțiune dedicată includerii de biblioteci	-- Exemplu de circuit logic library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. Aici se declară intrările și ieșirile circuitului precum și tipul de date vehiculate prin acestea. În cazul de față: - numele entității este: clc1 - avem 3 intrări notate a, b, c și o ieșire notată yout ; - toate intrările și ieșirile sunt de tip <i>std_logic</i> , deci pot avea următoarele stări: H, L, X, HiZ	entity clc1 is port (a, b, c : in std_logic; yout : out std_logic); end clc1;
Secțiune dedicată descrierii arhitecturii. Aici se precizează efectiv schema sau funcționarea entității declarate anterior. În cazul de față: - numele arhitecturii este: arh_1 ; - asocierea arhitecturii se face cu entitatea clc1 ; - expresia algebrică a funcției logice este: $yout = \bar{c}\bar{b}a + c\bar{b}\bar{a} + cb\bar{a}$	architecture arh_1 of clc1 is begin -- o singura declaratie concurenta yout <= ((not c) and (not b) and a) or (c and (not b) and (not a)) or (c and b and (not a)); end arh_1;

Exemplul 2: O altă modalitate de descriere a funcției logice **yout**

Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de circuit logic library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. (Secțiune identică cu cea din exemplul anterior).	entity clc1 is port (a, b, c : in std_logic; yout : out std_logic); end clc1;
Secțiune dedicată descrierii arhitecturii. În această secțiune apar câteva modificări: - numele arhitecturii este: arh_2 ; - sunt declarate 3 semnale: s1, s2, s3 - sunt folosite 4 declarații concurente; Semnalul din limbajul VHDL are ca echivalent fizic un fir de legătură. Semnalele trebuie declarate între architecture și begin . Starea logică a ieșirii yout este reactualizată de fiecare data când apare o modificare la oricare din semnalele s1, s2, s3 . Semnalele declarate într-o arhitectură sunt recunoscute numai în interiorul acesteia.	architecture arh_2 of clc1 is signal s1, s2, s3 : std_logic; begin -- mai multe declaratii concurente s1 <= (not c) and (not b) and a; s2 <= c and (not b) and (not a); s3 <= c and b and (not a); yout <= s1 or s2 or s3; end arh_2;

Exemplul 3: Utilizarea unui proces pentru descrierea funcției **yout**

Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de circuit logic library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. (Secțiune identică cu cea din exemplul anterior).	entity clc1 is port (a, b, c : in std_logic; yout : out std_logic); end clc1;
Secțiune dedicată descrierii arhitecturii. - În descriere se folosește un singur proces denumit p1. Atribuirea unui nume pentru proces este opțională. - Activarea procesului (altfel spus, lansarea în execuție) se face pentru orice eveniment apărut pe intrările a, b, c. - Se recomandă ca în proces să existe declarații pentru ambele valori logice ale ieșirii. - Declarațiile de tip if – then nu pot fi folosite decât în interiorul unui proces. - În lista de sensibilități a procesului trebuie incluse toate semnalele ce apar în partea dreaptă a declarațiilor din interiorul procesului. - Este necesar ca rezultatul evaluării condiției de test din paranteza lui if să fie TRUE sau FALSE, de aceea se folosește c='0' and b='1' și nu not c and b.	architecture arh_3 of clc1 is begin p1 : process (a, b, c) begin yout <= '0'; if ((c='0') and (b='0') and (a='1')) then yout <= '1'; end if; if ((c='1') and (b='0') and (a='0')) then yout <= '1'; end if; if ((c='1') and (b='1') and (a='0')) then yout <= '1'; end if; end process p1; end arh_3;
Descrierea arhitecturii - variantă alternativă. O altă posibilitate de descriere a arhitecturii funcției binare yout se bazează pe utilizarea unei declarații de tip if – then – else .	architecture arh_3_bis of clc1 is begin p2 : process (a, b, c) begin if (((c='0') and (b='0') and (a='1')) or ((c='1') and (b='0') and (a='0')) or ((c='1') and (b='1') and (a='0'))) then yout <= '1'; else yout <= '0'; end if; end process p2; end arh_3_bis;

Exemplul 4: Utilizarea declarației de tip **when – else** pentru descrierea funcției logice **yout**

Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de circuit logic library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. (Secțiune identică cu cea din exemplul anterior).	entity clc1 is port (a, b, c : in std_logic; yout : out std_logic); end clc1;
Secțiune dedicată descrierii arhitecturii. - numele arhitecturii este: my_clc4; - utilizarea declarației when-else trebuie făcută astfel încât să apară ambele valori logice ale ieșirii; - Este necesar ca rezultatul evaluării condiției de test din paranteza lui when să fie TRUE sau FALSE, de aceea se folosește c='0' and b='1' și nu not c and b.	architecture my_clc4 of clc1 is begin yout <= '1' when ((c='0' and b='0' and c='1') or (c='1' and b='0' and c='0') or (c='1' and b='1' and c='0')) else '0'; end my_clc4;

Exemplul 5: Utilizarea vectorilor în descrierea funcției logice **yout**

În limbajul VHDL, prin declarația **vector** se introduc magistralele, fie de intrare fie de ieșire. Spre exemplu, prin declarația **d_out : out std_logic_vector (3 downto 0);** se descrie o magistrală de ieșire de 4 biți în care **d_out(3)** este cel mai semnificativ bit. După cum se va vedea în acest exemplu, utilizarea vectorilor simplifică destul de mult descrierea circuitelor digitale.

Există mai multe modalități de atribuire a stării logice a unei magistrale de date:

d_out <= "1100";	-- atribuirea tuturor biților din magistrală
d_out(3) <= '1';	-- atribuirea unui singur bit din magistrală (bitul D3)
d_out(2 downto 1) <= "1100";	-- atribuirea unei porțiuni din magistrală (biții D2 și D1)

Pentru utilizarea vectorilor în implementarea funcției binare **yout** este necesar să redenumim intrările circuitului combinațional astfel încât acestea să aparțină unei magistrale de intrare de 3 biți.

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>-- Exemplu de circuit logic library IEEE; use IEEE.std_logic_1164.all;</pre>
Secțiune dedicată descrierii entității. În acest exemplu se folosește un vector de intrare format din 3 biți, d_in , prin care sunt introduse cele 3 intrări ale circuitului logic. Am folosit următoarea asociere : - d_in(2) pentru intrarea a ; - d_in(1) pentru intrarea b ; - d_in(0) pentru intrarea c ;	<pre>entity clc1 is port (d_in:in std_logic_vector(2 downto 0); yout : out std_logic); end clc1;</pre>
Secțiune dedicată descrierii arhitecturii. (varianta 1)	<pre>architecture my_clc5 of clc1 is begin yout <= '1' when ((d_in <= "001") or (d_in <= "100") or (d_in <= "100")) else '0' ; end my_clc5;</pre>
Secțiune dedicată descrierii arhitecturii. (varianta 2)	<pre>architecture my_clc5_bis of clc1 is begin p3 : process (d_in) begin yout <= '0' ; if ((d_in = "001") or(d_in = "100")or(d_in = "100")) then yout <= '1' ; else yout <= '0' ; end if; end process p3; end my_clc5 bis;</pre>

◆ **Exemplul 6:** Descrierea comportamentală a unui circuit logic.

Una dintre cele mai puternice facilități oferite de VHDL o constituie descrierea comportamentală a unui circuit logic. Această facilitate permite proiectantului să dezvolte sisteme logice foarte complexe fără a fi preocupat de schema logică detaliată, ci doar de descrierea comportamentală de nivel înalt a respectivului circuit.

Spre exemplu, un circuit de adunare a două numere reprezentate pe 3 biți fiecare, se poate descrie printr-o singură declarație de forma $z \leq x + y$. Sarcina sintetizării scheme logice a respectivului sumator cade în sarcina mediului software în care se dezvoltă aplicația.

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre>-- Exemplu de CLC descris comportamental library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_unsigned.all; use IEEE.std_logic_arith.all;</pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: sum - avem 2 magistrale de intrare de câte 3 biți; - o ieșire z pe 5 biți ; - toate intrările și ieșirile sunt de tip <i>std_logic</i> .	<pre>entity sum is port(x,y:in std_logic_vector(2 downto 0); z:out std_logic_vector(4 downto 0)); end sum;</pre>
Secțiune dedicată descrierii arhitecturii.	<pre>architecture arh_sum of sum is begin z <= x + y; end arh_sum;</pre>



3. Desfășurarea lucrării

3.1. Implementarea funcțiilor logice folosind rețele de porți (Metoda clasică)

- A.** Pentru circuitul logic cu o intrare și o ieșire având tabelul de adevăr prezentat în tabelul 3 se cere:
- implementarea funcției **yout** folosind forma canonică disjunctivă și un număr cât mai redus de porți;
 - implementarea funcției **yout** folosind forma canonică conjunctivă și un număr cât mai redus de porți;
 - implementarea funcției **yout** folosind doar porți NAND;
 - implementarea funcției **yout** folosind doar porți NOR;
- B.** Pentru circuitul logic cu 3 intrări și 4 ieșiri ce realizează funcția $Y=X+3$ (vezi tabelul 1 din această lucrare) se cere:
- implementarea funcțiilor logice folosind cealaltă formă canonică;
 - implementarea funcțiilor logice folosind doar porți NAND cu două intrări (vezi tabelul 2);
 - implementarea funcțiilor logice folosind doar porți NOR;

3.2. Utilizarea ISE WebPack pentru descrierea aplicațiilor sub formă de scheme logice

- A.** Implementarea și verificarea pe macheta de laborator cu CPLD a schemei logice din tabelul 1.
- B.** Implementarea și verificarea pe macheta de laborator cu CPLD a schemei logice din tabelul 2.
- C.** Implementarea și verificarea pe macheta de laborator cu CPLD a schemei logice din figura 8.

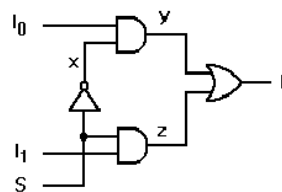


Fig.8

Mod de lucru: -vezi secțiunea 4 (Indicații privind modul de lucru).

3.3. Implementarea CLC-urilor folosind limbajul VHDL

Partea I: Descrierea structurală (pornind de la o schemă logică impusă)

- A.** Pentru schema logică din tabelul 1 se cere:
- O descriere în limbaj VHDL folosind modul de lucru din exemplu 1.
 - O descriere în limbaj VHDL folosind modul de lucru din exemplu 2.
- Pentru fiecare caz în parte realizați o implementare pe macheta de laborator cu CPLD și verificați dacă circuitul implementat funcționează corect.
- B.** Pentru schema logică din tabelul 2 se cere:
- O descriere în limbaj VHDL folosind modul de lucru din exemplu 1.
 - O descriere în limbaj VHDL folosind modul de lucru din exemplu 2.
- Pentru fiecare caz în parte realizați o implementare pe macheta de laborator cu CPLD și verificați dacă circuitul implementat funcționează corect.

Mod de lucru: -vezi secțiunea 4 (Indicații privind modul de lucru).

Partea a II-a: Descrierea structurală (pornind de la tabelul de adevăr)

- C.** Pornind de la tabelul de adevăr al clc-ului din tabelul 1, se cere:
- O descriere în limbaj VHDL folosind modul de lucru din exemplu 3.
 - O descriere în limbaj VHDL folosind modul de lucru din exemplu 4.
 - O descriere în limbaj VHDL folosind modul de lucru din exemplu 5.

Pentru fiecare caz în parte realizați o implementare pe macheta de laborator cu CPLD și verificați dacă circuitul implementat funcționează corect.

- D.** Pentru un circuit logic combinațional cu 5 intrări și o ieșire, ce realizează funcția de vot majoritar (ieșirea este în "1", atunci când la intrări predomină valoarea logică "1") se cere:
- tabelul de adevăr redus;
 - programul VHDL folosind modul de lucru din exemplul 1;
 - verificarea programului pe macheta de laborator;

Mod de lucru: -vezi secțiunea 4 (Indicații privind modul de lucru).

Partea a III-a: Descrierea comportamentală

- E.** Folosind descrierea comportamentală (similară exemplului 6), se cere descrierea și verificarea pe macheta de laborator a unui circuit de sumare a două numere exprimate pe 4 biți.
- F.** Folosind descrierea comportamentală (similară exemplului 6), se cere descrierea și verificarea pe macheta de laborator a unui circuit de comparare a două numere exprimate pe 4 biți.

Mod de lucru: -vezi secțiunea 4 (Indicații privind modul de lucru).

4. Indicații privind modul de lucru

Pentru fiecare aplicație este necesară deschiderea unui nou proiect după metodologia prezentată în secțiunea 2.3.1. a prezentei lucrări de laborator.

Toate aplicațiile din această lucrare necesită doar un singur fișier sursă (fie schemă logică, fie sursă VHDL) și un singur fișier de constrângeri.

Referitor la conectarea intrărilor și a ieșirilor din circuit facem următoarele precizări:

- Variabilele de intrare se vor conecta la switch-urile (comutatoare cu două poziții) de pe macheta de laborator. În acest mod, trecerea comutatorului de pe o poziție pe alata echivalează cu schimbarea stării logice a variabilei de intrare. Deschiderea.
- Variabilele de ieșire se vor conecta la LED-urile de pe macheta de laborator. În acest mod, în momentul în care o variabilă de ieșire este în unu logic, LED-ul asociat luminează.
- În cazul aplicațiilor de sumatoare sau comparatoare binare, fiecare număr de intrare respectiv ieșire va fi reprezentat printr-un vector. Un vector de intrare trebuie conectat la un pachet de switch-uri (un număr corespunzător de switch-uri ce sunt amplasate unul lângă altul) iar vectorul de ieșire va fi reprezentat pe un pachet de LED-uri.

NET "x<0>" LOC = "P37";
NET "x<1>" LOC = "P40";
NET "x<2>" LOC = "P43";

NET "y<0>" LOC = "P54";
NET "y<1>" LOC = "P52";
NET "y<2>" LOC = "P50";

NET "z<0>" LOC = "P75";
NET "z<1>" LOC = "P71";
NET "z<2>" LOC = "P67";
NET "z<3>" LOC = "P65";
NET "z<4>" LOC = "P62";

Fișierul de constrângeri pentru exemplul 6 este prezentat alăturat. Pentru introducerea numărului x s-au folosit primele trei comutatoare (SW1, SW2, SW3), iar pentru y s-au folosit ultimele trei comutatoare (SW6, SW7, SW8). Afișarea rezultatului se face pe primele 5 LED-uri (LD1÷LD5).

